

Hash Tables and Collision Handling

Hash Table Properties

- hash table: array that contains (key, value) pairs
- table size: current capacity (array length)
- load factor (LF): $\frac{(\textit{number of key,value pairs in table})}{(\textit{table size})}$

Table Size and Collisions

- Experiment repeated 10 times per table size:
 - 100 random integer keys
 - “hash function”: abs(key) % (table size)

How often do collisions occur for different table sizes?

# keys	table size	load factor	# of collisions
100	<u>10,000</u>	<u>0.01</u>	<u>0</u> or <u>1</u>
100	<u>1,000</u>	<u>0.1</u>	<u>3-7</u>
100	<u>100</u>	<u>1</u>	<u>35-47</u>
100	<u>10</u>	<u>10</u>	<u>90</u>

Resizing: When?

- When the table is “full”:

$$load_factor \geq load_factor_threshold$$

- Who defines the threshold?

Let user of our data structure set it, with good default (.7-.8).

Resizing: Complexity

- Resizing: $O(1)$
- Rehashing: $O(N)$, where N is the number of keys in the old hash table
- Amortized over many inserts: $O(1)$

Collision Handling: Open Addressing

- Each element of hash table stores at most one key, value pair
- If there is a collision, look for next “open address”

Open Addressing: Linear Probing

- Probe sequence: $H_K, H_K + 1, H_K + 2, H_K + 3, \dots$
- H_K = *hash index*

key	seq
<u>166</u>	1 ✓
<u>359</u>	7, 7+1=8 ✓
263	10, 10+1=11(σ), 1, 2, 3, 4, 5 ✓

hash function: key % (table size)

↓	↓↓	↓	↓	↓	↓		↓	↓		↓
0	1	2	3	4	5	6	7	8	9	10
440	166	266	124	246	263		337	359		351

Open Addressing: Look Up

- Look Up: 263, 330

0	1	2	3	4	5	6	7	8	9	10
440	166	266	124	246	263		337	359		351

- Delete 266, Look Up: 263

0	1	2	3	4	5	6	7	8	9	10
440	166	DEL	124	246	263		337	359		351

Open Addressing: Quadratic Probing

- Add polynomials of order 2: $H_K + \underline{c_0}i + \underline{c_1}i^2$ (usually: $c_0 = \underline{0}$, $c_1 = \underline{1}$)
- i : step number
- Probe sequence: $\underline{H_K}, \underline{H_K + 1^2}, \underline{H_K + 2^2}, \underline{H_K + 3^2}, \dots$

key	seq
166	1 ✓
359	7, $7+1^2=8$ ✓
<u>263</u>	$10, 10+1^2=11(0), 10+2^2=14(3), 10+3^2=19(8), 4, 2, 2, 4, 8, 3,$ $0, 10, 0, 3, \dots$

hash function: key % (table size)

↓	↓		↓				↓	↓		↓
0	1	2	3	4	5	6	7	8	9	10
440	166	266	124	246			337	359		351

Open Addressing: Double Hashing

- Second hash function for step size (s)

$$\underline{s} = \underline{\text{hash2}(\text{key})}$$

↳ 2nd indep. hash function

- Probing sequence: H_K , $H_K + s * 1$, $H_K + s * 2$, $H_K + s * 3$, ...

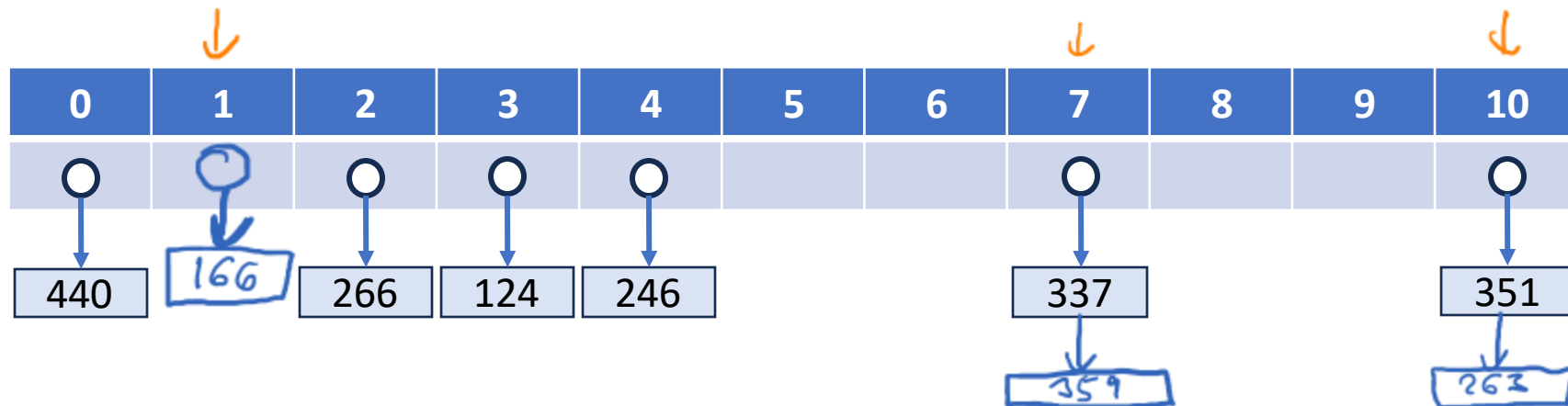
Collision Handling: Chaining

Each element of the hash table is a “chain” that can hold multiple (key, value) pairs.

↳ linked lists

key	seq
<u>166</u>	1 ✓
<u>359</u>	7 ✓
<u>263</u>	10 ✓

hash function: $\text{key} \% (\text{table size})$



Complexities

$N \hat{=}$ # key, value pairs
in table

	worst case	average case	best case
insert (<u>put</u>)	$O(N)$	$O(1)$	$O(1)$
look up (<u>get</u>)	$O(N)$	$O(1)$	$O(1)$
remove	$O(N)$	$O(1)$	$O(1)$